AI301 | AI Open Source Capstone

AI Open Source Capstone Fall 2026 (@ Section 1 | Wednesdays 3PM - 5PM PT)
Personal Member ID#: 92542

Need help? Post on our class slack channel or email us at support@codepath.org

Welcome

Getting Started

Candidate Issues

▶ Course Info

AI Prompting Guide

Course Progress

▼ Beat 1

Unit 1

Unit 2

Unit 3

Unit 4

▶ Beat 2

▶ Beat 3

Schedule

Overview | Activity | Assignment | Report

Assignment 2: Claim and Reproduce

Submit

> *A stranger could re-run it. The evidence shows it. You said what happened.*

🛠️ The work, in order

Steps 1-3 build and calibrate your skill, 4-6 use it on your real issue, and 7 submits.

1. **Make your skill runnable**: clone your Unit 2 materials (`git clone https://github.com/codepath/ai301-unit2-starter.git`, or Code, then Download ZIP on the repo page), then copy the whole `skill/` folder from the clone to `~/.claude/skills/repro-check/` (create the folders if they do not exist). This is how a Claude Code skill is installed: `claude` finds any skill under `~/.claude/skills/` from whatever directory you run it in.

2. **Finish the three components**: your `rubric.md` (with your worksheet changes from the activity folded in), your `references/evidence-guide.md`, and your `voice-guide.md`, all in the skill's template formats, all edited **inside your installed copy** at `~/.claude/skills/repro-check/`. That copy is your one canonical set: point the harness's `--rubric` and `--evidence` flags at those same files, so eval runs and live runs always grade with the same judgment.

3. **Run the harness** from the clone's `eval/` directory. If you want a warm-up first, grade `eval/packages/calib-02.md` by hand with your revised rubric; it spends no credit and shakes out wording you just changed. Then one full eval run, then read every disagreement with the gold labels and re-run just those packages with `--only` while you revise (the eval README in your Unit 2 materials has both commands and the canary rule for revisions that loosen a check). Repeat until you clear the bar (18 of the 20 scored packages agreeing with the gold labels, with every category matched; **The points** below says why 18), then add `--save-run eval-run.txt` to one confirming full run; that harness-written file is part of what you submit.

4. **Claim your issue**: draft your claim comment, run the skill in **live mode** on the draft, revise until it passes, then post it on your issue. To run live mode: from the folder holding your draft, run `claude "repro-check: grade my draft claim comment in <file> for issue <URL>"`. A correct run reads your installed skill's files, prints a short per-check summary (the repro checks report as not yet applicable on a claim-only draft), and ends with a fenced JSON block whose verdict is `accept` or `reject`; if you do not see that block, the skill did not run. Your claim goes up **before** you reproduce: it names your issue and what you will do next, and promises the report. The lecture's claim comment shows the after-the-fact form, written by an author who had already reproduced; yours comes first, so promise, do not assert.

5. **Fork Path Review, then clone your fork**: open the Path Review repo named in your skill's `scope.md` and press **Fork** at the top right, taking the defaults. That gives you your own copy of the repo under your account. Clone that copy rather than the original (`git clone <your fork's URL>`), and work in it from here on: you have no push access to the shared repo, so the branch you push next week and the pull request that follows both come from this fork. If a push later asks you for a password, GitHub wants a personal access token rather than your account password; a TF can get you through it.

6. **Reproduce your issue**: set up the sandbox repo environment from the repo's own docs, reproduce the bug, and write the repro report. Run the skill in live mode on the full package (claim comment + report), the same way as step 4; when it says ready, post the repro comment.

7. **Submit**: everything lands in your course repo, in the three places listed under **What you submit** below. Then submit your repo link on this tab.

Path Review house rules for the claim beat (your skill's `scope.md` carries the full statements): a classmate's claim does not block you, post your own claim anyway; never piggyback a repro ("same as above, can confirm"), your proof goes up in your own words even on a shared issue; credit attaches to the work you post, not to being first.

📦 What you submit

Your course repo link, through the course portal. It is the same link as unit 1, with this unit's additions in place.

Two repos are in play this unit, and their layouts do not mirror each other. Your **Unit 2 materials** (the clone from step 1) hold the skill under `skill/` and the harness under `eval/`; nothing you submit lives there. Your **course repo** (the one you created from the course template in unit 1) is where the work lands, and its `tools/` and `beat-1-sandbox/` folders are already in place from the template.

▶ ❗ **IMPORTANT: Submit a link to your entire repo, not just a folder.** *(What does this mean?)*

Three places in it carry this unit's work:

1. **`tools/repro-check/`**: the course repo's folder for this unit's skill; the template already has it, holding only a placeholder README. It sits alongside unit 1's `tools/issue-select/` and holds your filled `rubric.md`, `references/evidence-guide.md`, and `voice-guide.md`, plus the skill's `SKILL.md` and `scope.md`. Upload the current files from your installed copy (`~/.claude/skills/repro-check/`) when you submit; the web upload flow from unit 1 works the same way.

2. **`beat-1-sandbox/unit-2/reproduction.md`**: the links to your claim comment and your repro comment on your issue (or the house issue), **each with the text of that comment pasted under its link**, plus its short reflection prompts. Both comment rows are graded from the pasted text, so post the comment and then copy what you posted into the same field. It also carries the four eval-iteration fields, **Run history**, **Package analysis**, **Check rationale**, **Trade-offs**, the graded account of how your tool reached the run in `eval-run.txt`. Each is scored on its own.

3. `beat-1-sandbox/unit-2/eval-run.txt`: one complete eval run (run from the `eval/` directory of your Unit 2 materials), written by the harness rather than by you. Add `--save-run eval-run.txt` to your confirming full run and it writes the file itself, under a header recording the pinned model, the tool it graded, and a fingerprint of each file that went into the run; upload that file with the same web flow. A partial `--limit` or `--only` run refuses to write it. Never hand-edit it: the account of the runs it took belongs in your write-up.

🎯 The points

This assignment is worth **25 points**. The full rubric is on the **Grading Rubrics** page under Course Info; every graded criterion is on it. In short: the skill you uploaded carries 8 points, the eval run and write-up 10, and your posted comments 7. The 10 eval points split two ways:

- **3** for committing one complete run, written by the harness. A partial run, or none, earns 0.
- **7** across four separate fields in `beat-1-sandbox/unit-2/reproduction.md`, each scored on its own: **Run history** in order (2), **Package analysis** naming one scored package by id with both verdicts and your reasoning (2), **Check rationale** quoting the check as it now reads with the quote matching the file you uploaded (2), and **Trade-offs** (1).

A first run that already agrees on all 20 is a complete answer, and loses nothing. The item you examine does not have to be one your rubric got wrong: name a scored item, say what your rubric decided and what the gold label said, and explain why it read it that way. Getting it right and knowing why is the whole answer. The score is for the account, not for having struggled.

Why the bar is 18 of 20. It is the number to aim at, and it sets no points by itself: the 10 eval points above score the run you commit and your account of it, not the tally. 16 of the 20 scored packages carry loud signals a reasonable rubric should never miss, and 4 are genuinely arguable calls. The **category floor** has teeth this week: the set contains exactly one disclosure-wall package (a repo whose stated policy requires disclosing AI assistance, and comments that do not disclose), and a rubric with no conventions check cannot buy that miss back on volume. The 4 calibration packages from the activity are never scored. An unmatched category is the best thing you can write about in your iterations: it names a surface your rubric cannot see.

The comment points ask for the things this week taught: a claim that names your issue's specifics, and a repro report a stranger could re-run, with the environment recorded and the observed behavior shown. Both are matched to you by the **GitHub username** you record in the phase file, so fill that in. (Promising only your investigation, never a fix or a date, is still the right way to claim; the voice guide is where that habit is graded.) **An honest, evidenced cannot-reproduce earns full points**: reporting that faithfully is exactly what phase 2 is for. A confident reproduction of the wrong behavior does not. On the house issue, the same criteria grade the same things.

🔁 Retries

Unlimited before the deadline. A full eval run costs about $4 of your course credit; the revise loop should not. After a full run, re-grade only the packages you disagreed on with `--only pkg-07,pkg-12` (about $0.20 per package), and save full runs for confirming a rubric you believe in. Partial runs never count as the submitted run, and they cannot show the category tallies. A

revision that loosens a check can flip a package that agreed before, so when you loosen, add a canary to your `--only` list (one package from each single-package category your change could touch; the eval README shows the move) before you spend a confirming full run. For your comments, run your skill in live mode on your drafts before posting; posting a comment you never checked with your own tool is the avoidable failure mode.

⏰ Deadline

Due: **Monday, September 28th at 2:59AM EDT**

Unit 3 builds on your reproduced issue: the fix loop starts from the proof you post this week, so late work costs you twice.